

RAPPORT DE STAGE

Système de reconnaissance gestuelle pour TIAGO **(27/04/2026 ~ 30/06/2026)**

CESI école d'ingénieurs
Rouen, Saint- Étienne-du-Rouvray
RAGOT Nicolas

NAM Seohyun
PGE – CPI A2 Info
ROUSSEL Antoine

Mode de diffusion :

Restreinte ☐

Libre ☒

Résumés : français et anglais

Ce projet porte sur la reconnaissance gestuelle dans le cadre de l'interaction humain-robot. L'objectif est de permettre à un utilisateur de communiquer avec le robot TIAGO à l'aide de gestes réalisés devant une caméra.

Pour réaliser ce projet, nous avons étudié plusieurs solutions de reconnaissance gestuelle et choisi d'utiliser MediaPipe pour détecter les mains et récupérer les points caractéristiques des doigts. Nous avons ensuite utilisé l'algorithme KNN afin de reconnaître les gestes à partir d'exemples enregistrés dans un fichier de données.

Plus de 35 000 exemples de gestes ont été collectés et utilisés pour entraîner le modèle. Une étape de normalisation a également été mise en place afin de rendre la reconnaissance moins sensible à la taille de la main et à la distance par rapport à la caméra.

Les tests réalisés ont permis de reconnaître plusieurs gestes en temps réel et d'afficher un émoji correspondant au geste détecté. Les résultats obtenus sont satisfaisants mais des améliorations restent encore possibles, notamment pour mieux reconnaître certains gestes très proches et pour intégrer directement les commandes au robot TIAGO.

This project focuses on gesture recognition in the context of human-robot interaction. The objective is to allow a user to communicate with the TIAGO robot using hand gestures performed in front of a camera.

To carry out this project, several gesture recognition solutions were studied, and MediaPipe was chosen to detect hands and extract hand landmark points. The K-Nearest Neighbors (KNN) algorithm was then used to recognize gestures from examples stored in a dataset.

More than 35,000 gesture samples were collected and used to train the model. A normalization step was also implemented to make the recognition less sensitive to hand size and distance from the camera.

The tests carried out made it possible to recognize several gestures in real time and display an emoji corresponding to the detected gesture. The results obtained are satisfactory, but improvements are still possible, particularly for distinguishing very similar gestures and for directly integrating gesture commands with the TIAGO robot.

Sommaire

Introduction.....	5
Développement	5
1. Bibliothèques	5
1.1 Externe	5
1.1.1 MediaPipe.....	5
1.1.2 Scikit-learn.....	6
1.1.3 Joblib	7
1.1.4 Pandas	7
1.1.5 NumPy.....	7
1.1.6 OpenCV	8
1.2 Interne.....	8
1.2.1 CSV	8
1.2.2 OS	8
1.2.3 Math	9
2. Machine Learning	9
2.1 Algorithme KNN.....	9
2.2 Data augmentation.....	10
2.3 Normalisation	11
2.4 Zero Padding.....	13
3. Conception du système	14
3.1 Pipeline d'apprentissage.....	15
3.2 Pipeline de fonctionnement en temps réel	16
3.3 Schéma global de la chaîne de traitement.....	17
4. Expérimentations et résultats	18
5. Limites et perspectives d'améliorations	25
Conclusion	26

Liste des figures

Figure 1.....	6
Figure 2.....	8
Figure 3.....	9
Figure 4.....	9
Figure 5.....	10
Figure 6.....	10
Figure 7.....	11
Figure 8.....	12
Figure 9.....	12
Figure 10.....	14
Figure 11.....	15
Figure 12.....	16
Figure 13.....	17
Figure 14.....	18
Figure 15.....	19
Figure 16.....	20
Figure 17.....	21
Figure 18.....	22
Figure 19.....	24

Liste des tableaux

Tableau 1	23
Tableau 2.....	23

Introduction

Dans le cadre de l'industrie 5.0 et de l'interaction humain-robot, ce projet a pour objectif de développer un système de reconnaissance gestuelle permettant à un opérateur de communiquer avec le robot TIAGO à travers des gestes simples réalisés devant une caméra.

Le livrable 1 de ce projet consiste à étudier et développer une application de reconnaissance gestuelle en temps réel. Cette première phase comprend un état de l'art des solutions existantes ainsi que la réalisation d'une maquette fonctionnelle en Python basée sur la détection des mains et le Machine Learning.

L'application développée permet de reconnaître différents gestes réalisés devant une caméra et d'afficher un émoji correspondant au geste détecté. Plusieurs expérimentations ont été réalisées afin d'évaluer les performances et la fiabilité du système.

Ce rapport présente les notions étudiées lors de la phase de recherche, leur application dans le projet, les expérimentations réalisées et les résultats obtenus. Il se termine par une analyse des limites du système et des perspectives d'amélioration.

Ce travail constitue une première étape avant une future intégration de la reconnaissance gestuelle au robot TIAGO.

Développement

Dans le cadre du développement du système de reconnaissance gestuelle, nous avons utilisé plusieurs bibliothèques et la technique du machine learning afin de permettre la détection des gestes, le traitement des données et l'affichage des résultats en temps réel.

1. Bibliothèques

1.1 Externe

1.1.1 MediaPipe

MediaPipe est une bibliothèque open-source développée par Google qui permet de traiter en temps réel des données multimédias comme la vidéo ou les images afin de détecter et analyser des éléments du corps humain comme les mains, le visage ou la posture.

Dans notre projet, MediaPipe a été utilisée pour reconnaître les mains et suivre leurs mouvements grâce à des points de repère précis. Elle fonctionne avec une caméra et permet d'interpréter les gestes en direct.

(Commande d'installation : `$ python -m pip install mediapipe`)

Nous utilisons le modèle « HandLandmarker » qui permet de détecter 21 points clés sur chaque main. Ces 21 points correspondent aux articulations principales de la main (poignet, doigts, phalanges) ce qui permet d'analyser précisément la posture de la main et d'identifier les gestes réalisés.

Figure 1



(Source :

https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer#hand_landmark_model_bundle)

Par exemple :

- L'index est considéré comme replié si la distance entre les points 0 et 8 est inférieure à celle entre les points 0 et 6.
- Le majeur est considéré comme replié si la distance entre les points 0 et 12 est inférieure à celle entre les points 0 et 10.
- L'annulaire est considéré comme replié si la distance entre les points 0 et 16 est inférieure à celle entre les points 0 et 14.

1.1.2 Scikit-learn

Scikit-learn est une bibliothèque de machine learning qui fournit de nombreux algorithmes prêts pour la classification, la régression et l'analyse de données.

Dans notre projet, Scikit-learn a été utilisé pour entraîner le modèle de reconnaissance gestuelle. Nous avons utilisé l'algorithme KNN pour classer les gestes à partir des coordonnées extraites par MediaPipe. La bibliothèque a également été utilisée pour

séparer automatiquement les données en ensembles d'entraînement (80%) et de test (20%) pour éviter le surapprentissage des données ainsi que pour calculer la précision du modèle après l'apprentissage.

1.1.3 Joblib

Joblib est une bibliothèque utilisée pour sauvegarder et charger des objets Python dans des fichiers. Elle est souvent utilisée dans les projets de machine learning afin d'enregistrer un modèle entraîné pour pouvoir le réutiliser ultérieurement sans devoir refaire l'entraînement.

Dans notre projet, Joblib a été utilisée pour sauvegarder le modèle KNN après son entraînement dans un fichier nommé « `gesture_model.pkl` » selon les données enregistrées dans un fichier CSV. Ce fichier est chargé lors de l'exécution du système de reconnaissance gestuelle afin de réaliser les prédictions en temps réel.

1.1.4 Pandas

Pandas est une bibliothèque spécialisée dans la manipulation et l'analyse de données sous forme de tableaux. Elle permet notamment de lire facilement des fichiers CSV et d'accéder aux données de manière structurée.

Dans notre projet, Pandas a été utilisée pour charger le fichier « `dataset.csv` » contenant les coordonnées des gestes enregistrés. La bibliothèque a également permis de séparer les données d'entrée (coordonnées des mains) des labels (noms des gestes) correspondant aux différents gestes avant l'entraînement du modèle KNN.

1.1.5 NumPy

NumPy est une bibliothèque dédiée au calcul numérique et à la manipulation de tableaux de données. Elle offre de nombreuses fonctions mathématiques rapides et optimisées.

Dans notre projet, NumPy a été utilisée principalement lors de la phase de reconnaissance en temps réel. La bibliothèque permet notamment de manipuler les coordonnées des mains, de traiter les probabilités retournées par le modèle KNN et de rechercher la probabilité maximale afin de déterminer si un geste est reconnu avec un niveau de confiance suffisant (70% dans notre projet).

1.1.6 OpenCV

La bibliothèque OpenCV (Open Source Computer Vision Library) a été utilisée pour :

- Accéder à la webcam
- Afficher le flux vidéo en temps réel
- Dessiner les points détectés sur les mains
- Afficher les résultats de reconnaissance gestuelle

(Commande d'installation : `pip install opencv-python`)

1.2 Interne

1.2.1 CSV

Le fichier CSV a été utilisé pour enregistrer les données récupérées lors de la phase d'apprentissage.

Chaque ligne du fichier contient les coordonnées des points détectés pour un geste précis ainsi que son label (nom du geste) correspondant. Ces données servent ensuite à entraîner le modèle de reconnaissance.

1.2.2 OS

La bibliothèque OS permet d'interagir avec le système d'exploitation et de gérer les fichiers ainsi que les dossiers du projet. Elle permet de récupérer des chemins de fichiers de manière automatique et compatible avec différents systèmes d'exploitation.

Dans notre projet, OS a été utilisé pour localiser les différents fichiers nécessaires au fonctionnement du système comme le fichier « dataset.csv », le modèle « gesture_model.pkl » et les images des émojis. Cela permet d'éviter de saisir manuellement les chemins complets et garantit que le programme fonctionne correctement quel que soit l'emplacement du projet sur l'ordinateur.

```
# Création du chemin
BASE_DIR = os.path.dirname(os.path.abspath(__file__)) #
DATASET_PATH = os.path.join(BASE_DIR, "dataset.csv") # C
MODEL_PATH = os.path.join(BASE_DIR, "gesture_model.pkl")
```

Figure 2

1.2.3 Math

La bibliothèque Math fournit différentes fonctions mathématiques utiles pour effectuer des calculs numériques.

Dans notre projet, Math a été utilisée lors de la phase de normalisation des coordonnées des mains. Plus précisément, elle permet de calculer la distance de référence en appliquant la formule de la distance euclidienne entre le poignet et le bout du majeur grâce à la fonction « `sqrt()` » (racine carrée) pour obtenir un point de référence de la main. Cette distance sert ensuite à normaliser les coordonnées des 21 points détectés par MediaPipe afin de rendre le système moins sensible à la taille de la main ou à son éloignement par rapport à la caméra.

```
# Distance de référence en appliquant la distance euclidienne (= taille de main)
hand_size = math.sqrt(
    (middle.x - wrist.x) ** 2
    + (middle.y - wrist.y) ** 2
    + (middle.z - wrist.z) ** 2
)
```

Figure 3

Distance euclidienne à deux dimensions :

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}.$$

Figure 4

2. Machine Learning

2.1 Algorithme KNN

Le principe de l'algorithme KNN (K-Nearest Neighbors) est un algorithme qui prend une décision en observant les k voisins les plus proches.

Il fait partie du machine learning supervisé. Cela signifie que l'on fournit d'abord à la machine des exemples avec leur bonne réponse afin qu'elle puisse apprendre à les reconnaître.

Dans notre projet, nous indiquons à la machine quel geste est réalisé lors de la collecte des données. Les coordonnées des mains détectées par MediaPipe sont enregistrées dans un fichier CSV avec le nom du geste correspondant. Par exemple, un geste pouce levé sera enregistré avec le label « THUMBS_UP ». Notre système utilise donc une approche d'apprentissage supervisé.

Cet algorithme joue un rôle essentiel dans notre projet. Lorsqu'un utilisateur réalise un geste devant la caméra, MediaPipe extrait les 21 points caractéristiques de la main (63 points pour une main et 126 points pour deux mains). Ces données sont ensuite normalisées puis envoyées au modèle KNN. Le modèle compare le geste détecté avec les données d'apprentissage enregistrées dans le fichier CSV en calculant la distance euclidienne entre les différentes observations. Les 10 gestes ($k = 10$) les plus proches sont ensuite sélectionnés puis un vote majoritaire est effectué afin de déterminer le geste final reconnu.

Distance euclidienne à trois dimensions :

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + (p_3 - q_3)^2}.$$

Figure 5

Pourquoi $k = 10$ dans notre projet ?

Si la valeur de k est trop petite, le modèle risque d'être influencé par des données mal détectées ou peu représentatives ce qui peut conduire à une mauvaise reconnaissance du geste.

À l'inverse, si k est trop grand, des données moins similaires au geste réalisé peuvent être prises en compte augmentant ainsi le risque d'erreur.

Pour notre projet, nous avons retenu $k = 10$ ce qui permet d'obtenir une reconnaissance des gestes à la fois fiable et stable.

```
model = KNeighborsClassifier(n_neighbors = 10)
```

Figure 6

2.2 Data augmentation

Afin d'améliorer les performances du modèle de reconnaissance gestuelle, une phase d'augmentation des données a été réalisée lors de la collecte des exemples. Pour chaque geste, plusieurs échantillons ont été enregistrés dans des conditions variées : positions différentes de la main, orientations légèrement différentes et distances variables par rapport à la caméra.

Chaque exemple a été enregistré dans le fichier « dataset.csv » grâce à une action de sauvegarde manuelle. Au total, les données contiennent 36 794 échantillons ce qui représente 36 794 captures de gestes utilisées pour l'apprentissage du modèle.

Si le fichier CSV contient 36 794 échantillons, les données originales sont conservées et 3 nouvelles versions sont créées pour chaque échantillon. Chaque donnée d'origine génère au total quatre données d'entraînement (1 originale + 3 modifiées). Le nombre total d'échantillons augmente de 36 794 à 147 176, soit 4 fois plus.

Les 3 types de modifications sont appliqués pour créer ces nouvelles données. Tout d'abord, nous ajoutons un léger bruit aléatoire pour simuler les petites erreurs de la caméra (bruit gaussien). Ensuite, nous agrandissons ou réduisons légèrement la taille de la main afin de représenter différentes distances par rapport à la caméra. Enfin, nous faisons tourner légèrement la main pour reproduire différentes orientations possibles.

```
# On génère 3 variantes artificielles uniques pour cet échantillon donc fois 4 par rapport les données or
for _ in range(3):
    s = sample.copy() # copie les données dans toucher les données originals

    # 1. Ajout de bruit gaussien (simule l'imprécision/tremblement de la caméra)
    # On n'applique pas de bruit sur les valeurs masquées par le Zero Padding (égales à 0.0)
    mask = s != 0.0
    s[mask] = s[mask] + np.random.normal(0, 0.01, np.sum(mask))

    # 2. Variation d'échelle (simule la distance de la main par rapport à la caméra)
    scale = np.random.uniform(0.9, 1.1) # Variation de +/- 10%
    s = s * scale

    # 3. Rotation légère (simule l'inclinaison de la main de l'utilisateur)
    angle = np.random.uniform(-0.2, 0.2) # Entre environ -11° et +11°
    s = rotate_sample(s, angle)

    # Ajout de la version augmentée dans nos nouvelles listes
    X_aug.append(s)
    y_aug.append(current_label)
```

Figure 7

Cette quantité importante de données permet d'obtenir un ensemble d'apprentissage plus fiable et plus représentatif des situations réelles. Le modèle KNN dispose ainsi de nombreux exemples de référence pour comparer les gestes détectés en temps réel ce qui améliore la robustesse et la fiabilité de la reconnaissance.

2.3 Normalisation

La normalisation a été appliquée aux données afin de réduire les variations liées à la position de la main devant la caméra. Cette méthode joue aussi un rôle essentiel dans notre projet.

Elle permet de rendre la reconnaissance plus fiable face :

- Aux différences de taille des mains
- Aux distances variables par rapport à la caméra

- Aux légers déplacements de l'utilisateur

Le principe consiste à utiliser le poignet comme point de référence. Pour chaque main détectée, MediaPipe fournit les coordonnées des 21 points caractéristiques de la main comme nous avons déjà vu dans la partie MediaPipe.

D'abord, les coordonnées de chaque point sont recalculées par rapport au poignet (point 0) afin de repositionner à l'origine du repère (0, 0, 0).

```
# Parcourir les 21 landmarks (0 ~ 20)
for lm in hand_landmarks.landmark:

    # Pour chaque point : (Coordonnées du point actuel - Coordonnées du poignet) / Taille de la main
    x = (lm.x - wrist.x) / hand_size # (10 - 10) / 5 = 0 (exemple de poignet)
    y = (lm.y - wrist.y) / hand_size # (20 - 20) / 5 = 0 (exemple de poignet)
    z = (lm.z - wrist.z) / hand_size # (0 - 0) / 5 = 0 (exemple de poignet)
```

Figure 8

Ensuite, toutes les coordonnées sont divisées par une distance de référence correspondant à la taille de la main. Cette distance est calculée entre le poignet et le bout du majeur à l'aide de la distance euclidienne.

```
# Récupérer les deux points de référence
wrist = hand_landmarks.landmark[0] # poignet
middle = hand_landmarks.landmark[12] # extrémité du majeur

# Distance de référence en appliquant la distance euclidienne (= taille de main)
hand_size = math.sqrt(
    (middle.x - wrist.x) ** 2
    + (middle.y - wrist.y) ** 2
    + (middle.z - wrist.z) ** 2
)
```

Figure 9

Pour simplifier la démonstration, nous imaginons que la main soit représentée par seulement trois points :

- Poignet : (10 ; 20 ; 0)
- Bout du majeur : (10 ; 25 ; 0)
- Bout de l'index : (12 ; 24 ; 0)

D'abord, on calcule la distance entre le poignet et le majeur est utilisée comme taille de référence de la main :

- $\sqrt{(10 - 10)^2 + (25 - 20)^2 + (0 - 0)^2} = 5$ (= taille de référence)

Ensuite, la normalisation consiste à déplacer le poignet à l'origine (0 ; 0 ; 0) et diviser toutes les coordonnées par la taille de la main qui vaut 5.

On obtient :

- Poignet : $\frac{(10-10)}{5} = 0$; $\frac{(20-20)}{5} = 0$; $\frac{(0-0)}{5} = 0$, donc (0 ; 0 ; 0)
- Bout du majeur : $\frac{(10-10)}{5} = 0$; $\frac{(25-20)}{5} = 1$; $\frac{(0-0)}{5} = 0$, donc (0 ; 1 ; 0)
- Bout de l'index : $\frac{(12-10)}{5} = 0,4$; $\frac{(24-20)}{5} = 0,8$; $\frac{(0-0)}{5} = 0$, donc (0,4 ; 0,8 ; 0)

Les coordonnées finales normalisées sont donc [(0 ; 0 ; 0), (0 ; 1 ; 0), (0,4 ; 0,8 ; 0)].

Mais si une personne rapproche sa main de la caméra, toutes les coordonnées deviennent plus grandes. Par exemple, si la main est deux fois plus proche de la caméra :

- Poignet : (20 ; 40 ; 0)
- Bout du majeur : (20 ; 50 ; 0)
- Bout de l'index : (24 ; 48 ; 0)

La taille de référence devient alors 10 au lieu de 5. Après normalisation avec ces valeurs, on retrouve exactement les mêmes valeurs que nous avons déjà obtenu [(0 ; 0 ; 0), (0 ; 1 ; 0), (0,4 ; 0,8 ; 0)] même si les valeurs deviennent deux fois plus grandes.

L'intérêt est que même si l'utilisateur rapproche ou éloigne sa main de la caméra, les proportions restent identiques et les coordonnées normalisées conservent des valeurs très proches grâce à la division par la taille de la main.

Sans normalisation, le système pourrait considérer qu'il s'agit d'un geste différent.

Finalement, la normalisation améliore beaucoup la précision globale du système de reconnaissance gestuelle.

2.4 Zero Padding

Le Zero Padding (remplissage par des zéros) a été utilisé afin d'uniformiser la taille des données enregistrées dans le fichier CSV. Le modèle de machine learning nécessite que chaque ligne du fichier CSV possède exactement le même nombre de valeurs.

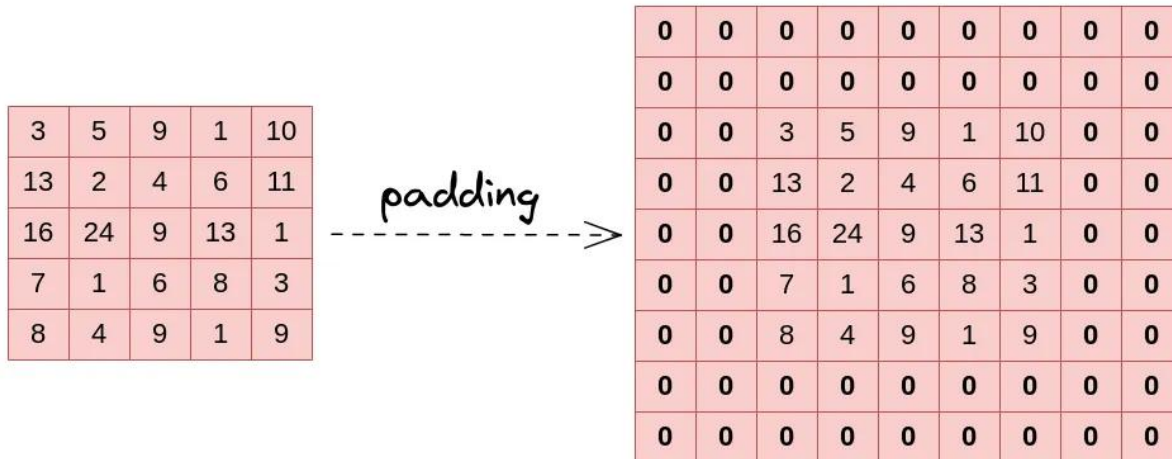
Chaque main détectée contient 21 points et chaque point possède trois coordonnées (x, y, z), soit :

- $21 \times 3 = 63$ valeurs pour un geste à une main
- $63 + 63 = 126$ valeurs pour un geste à deux mains

Afin de conserver une taille fixe pour toutes les données, les valeurs manquantes de la deuxième main sont remplacées par des zéros dans le cas pour une seule main.

- Geste à une main : 63 valeurs + 63 zéros = 126 valeurs
- Geste à deux mains : 63 + 63 = 126 valeurs

Figure 10



(Source : <https://www.baeldung.com/wp-content/uploads/sites/4/2023/02/padding.png>)

Cette technique permet d'éviter les erreurs lors de l'apprentissage et améliore la stabilité du modèle.

3. Conception du système

Nous avons utilisé un diagramme de séquence et un diagramme d'activité afin de représenter la conception globale de notre système. Nous avons réalisé deux diagrammes de séquence différents : le pipeline d'apprentissage et le pipeline de fonctionnement en temps réel. Et le diagramme d'activité montre le schéma global de la chaîne de traitement.

3.1 Pipeline d'apprentissage

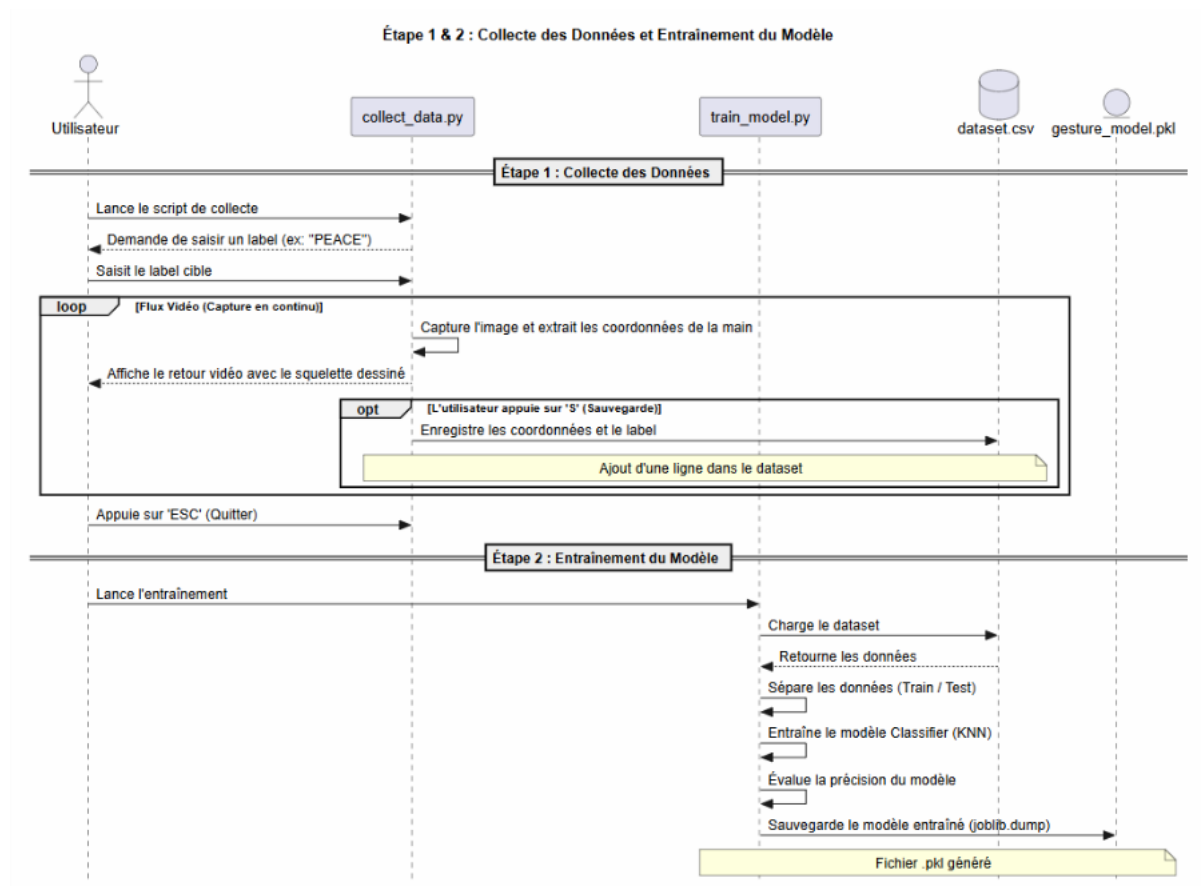


Figure 11

- **Collecte des données (collect_data.py)**

Il sert à créer notre propre base de données de gestes. Il utilise la webcam et la bibliothèque MediaPipe pour détecter la structure des mains en temps réel. Dès que l'utilisateur appuie sur la touche 'S', les données sont enregistrées dans le fichier « dataset.csv » avec le nom du geste associé.

- **Entraînement du modèle (train_model.py)**

Il prend en charge la phase d'apprentissage automatique (Machine Learning). Il charge le fichier « dataset.csv » précédemment créé et sépare les données pour l'entraînement (80%) et le test (20%). Il entraîne ensuite un algorithme KNN à reconnaître et associer les formes des mains aux bons gestes. Une fois l'apprentissage terminé, le modèle est sauvegardé sous la forme d'un fichier binaire (joblib) nommé « gesture_model.pkl ».

3.2 Pipeline de fonctionnement en temps réel

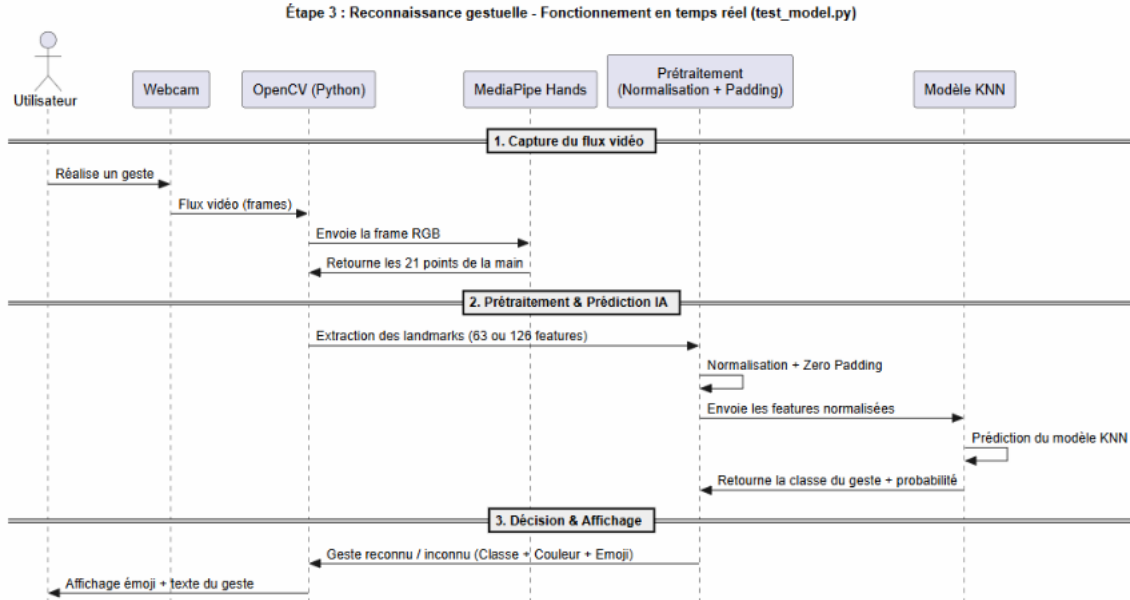


Figure 12

Le fonctionnement en temps réel décrit le processus allant du moment où l'utilisateur effectue un geste jusqu'à l'affichage du résultat sous forme de texte et d'emoji.

- **Reconnaissance en temps réel (test_model.py)**

Il charge le modèle entraîné « `gesture_model.pkl` » et analyse le flux vidéo de la webcam pour prédire le geste effectué en direct. Si le score de confiance dépasse le seuil de 70%, le script applique un lissage sur les mouvements pour stabiliser l'affichage puis superpose proprement le nom du geste en vert ainsi qu'un emoji personnalisé. Si le score de confiance ne dépasse pas le seuil de 70%, il affiche "Geste inconnu" en rouge.

3.3 Schéma global de la chaîne de traitement

Schéma global de la chaîne de traitement

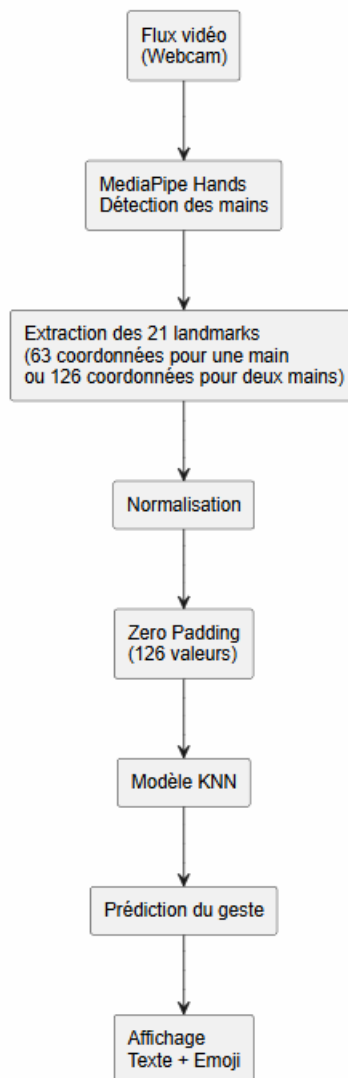


Figure 13

Ce schéma présente la chaîne de traitement utilisée lors de la phase de reconnaissance gestuelle. Le flux vidéo est capturé par la webcam puis traité par MediaPipe Hands afin d'extraire les coordonnées des points de repère des mains. Après une étape de normalisation et de mise en forme des données, les coordonnées sont transmises au modèle KNN entraîné. Le modèle effectue la classification du geste à partir des exemples présents dans le jeu de données et retourne la classe prédite ainsi qu'un niveau de confiance. Le résultat est ensuite affiché à l'utilisateur sous forme de texte et d'emoji en temps réel.

4. Expérimentations et résultats

Au début du projet, nous avons réalisé des tests avec seulement deux ou trois gestes simples sans utiliser de technique de Machine Learning. La reconnaissance était basée à coder la fonction par geste en calculant les distances entre les points de la main détectés par MediaPipe.

```
# ===== FONCTION : doigts levés =====  
def fingers_up(lm):  
  
    fingers = []  
  
    # Index (8 vs 6)  
    fingers.append(lm[8].y < lm[6].y)  
  
    # Middle (12 vs 10)  
    fingers.append(lm[12].y < lm[10].y)  
  
    # Ring (16 vs 14)  
    fingers.append(lm[16].y < lm[14].y)  
  
    # Pinky (20 vs 18)  
    fingers.append(lm[20].y < lm[18].y)  
  
    # Thumb (horizontal)  
    fingers.append(lm[4].x > lm[3].x)  
  
    return fingers
```

Figure 14

```
# ===== LOGIQUE GESTES =====

if fingers == [False, False, False, False, False]:
    gesture_text = "POING 🖐️ "

elif fingers == [True, True, True, True, True]:
    gesture_text = "MAIN OUVERTE 🖐️ "

elif fingers == [True, False, False, False, False]:
    gesture_text = "INDEX 🙌 "

elif fingers == [True, True, False, False, False]:
    gesture_text = "PEACE 🙌 "

else:
    gesture_text = "UNKNOWN"
```

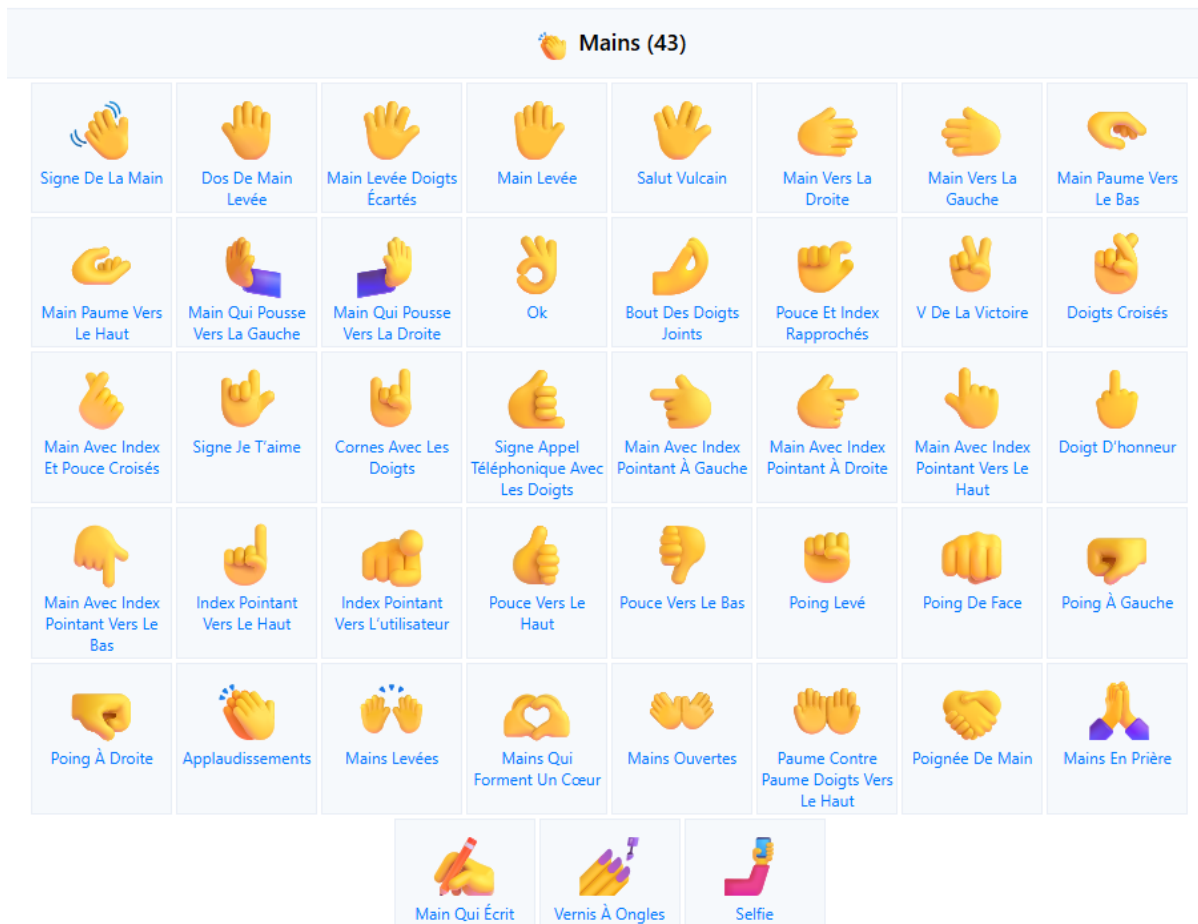
Figure 15

Cette méthode fonctionnait bien avec un petit nombre de gestes et nous a permis de comprendre le fonctionnement de la reconnaissance gestuelle. Cependant, lorsque nous avons voulu ajouter beaucoup plus de gestes, le code est devenu de plus en plus compliqué et même la performance n'était pas fiable. Il fallait définir des règles et des calculs de distance pour chaque geste ce qui demandait beaucoup de temps et rendait le programme difficile à maintenir.

D'après la recherche pour réaliser avec plusieurs gestes, nous avons trouvé la technique de la Machine Learning. Nous avons créé les données contenant de nombreux exemples de gestes au lieu de calculer la distance pour chaque geste. Le modèle a ensuite appris à reconnaître automatiquement les différents gestes à partir de ces exemples.

Voici les gestes que nous avons utilisé :

Figure 16



(Source : <https://emojiterra.com/fr/mains/>)

Afin d'évaluer et d'améliorer plus les performances de notre système de reconnaissance gestuelle, nous avons collecté et enrichi les données à partir de 6 mains appartenant à 3 personnes en les enregistrant sous différents angles, à différentes distances et dans différentes orientations de la main.

Pendant l'échange avec notre tuteur, il nous a donné l'idée de la normalisation des données. Nous avons alors commencé à collecter de nouvelles données pour cette version du système et nous avons comparé les résultats avec la version sans normalisation.

Le résultat était très intéressant. Avec l'ajout d'une seule fonction de normalisation, les performances de reconnaissance des gestes se sont améliorées. Ce qui nous a le plus surpris, c'est que cette nouvelle version obtenait de meilleurs résultats avec seulement 30 000 données alors que l'ancienne version utilisait plus de 50 000 données.

Après la démonstration du livrable 1 devant le directeur, nous avons suivi les conseils du directeur et du tuteur pour améliorer encore le système. Nous avons ajouté une technique de data augmentation afin d'apporter plus de variété aux données d'entraînement. Nous avons également augmenté la valeur de k de 3 à 10 pour afficher en temps réel le pourcentage de réussite de la reconnaissance d'un geste.

En effet, avec $k = 3$, seules trois valeurs de confiance étaient possibles :

- $3/3 = 100\%$
- $2/3 = 66\%$
- $1/3 = 33\%$

Avec $k = 10$, le pourcentage peut varier par incréments de 10% ce qui est plus fluide en visualisant.

Pour tester cette version améliorée, nous avons emprunté un écran télé pendant l'heure de midi et nous avons demandé à différentes personnes qui sont dans le hall du campus pour tester le système avec leurs mains.



Figure 17



Figure 18

Le résultat était très satisfaisant. Nous avons constaté une amélioration nette des performances par rapport à la version précédente.

Mais comment pouvons-nous savoir que le résultat est cohérent et repose sur des éléments factuels chiffrés ?

Tableau de répartition des échantillons par geste :

Geste	Nombre d'échantillons
Signe appel téléphonique avec les doigts	709
Doigts croisés	2162
Main avec index et pouce croisés	758
Bout des doigts joints	1011
Poing à gauche	57
Poing à droite	53
Poing de face	1244
Main vers la gauche	281
Main vers la droite	427
Mains qui forment un cœur	303
Cornes avec les doigts	1469
Signe « Je t'aime »	1174

Doigt d'honneur	512
Signe « OK »	1167
Mains ouvertes	471
Paumes jointes, doigts vers le haut	516
Main paume vers le bas	604
Main paume vers le haut	267
V de la victoire	1971
Pouce et index rapprochés	969
Index pointant vers l'utilisateur	932
Index pointant vers le bas	500
Index pointant vers la gauche	72
Index pointant vers la droite	107
Index pointant vers le haut	1549
Main avec index pointant vers le haut	1384
Poing levé	1283
Main levée	3218
Main levées	1296
Main levée doigts écartés	2710
Pouce vers le bas	969
Pouce vers le haut	1198
Salut vulcain	5451

Tableau 1

Tableau des statistiques globales :

Nombre total d'échantillons	36 794
Nombre de gestes	33
Moyenne d'échantillons par geste	1115
Geste le plus représenté	Salut vulcain (5451)
Geste le moins représenté	Poing à droite (53)

Tableau 2

En analysant les tableaux, nous pouvons remarquer que certains gestes ont été enregistrés plus de fois que d'autres comme les gestes Salut vulcain (5451), Main levée (3218), etc. Car certains gestes se ressemblent beaucoup et sont plus difficiles à reconnaître.

Par exemple, les gestes Salut vulcain, Main levée et Main levée doigts écartés ont une forme assez proche. De même, les gestes V de la victoire et Doigts croisés peuvent parfois être confondus.

Nous avons donc enregistré davantage d'exemples pour ces gestes afin que le modèle apprenne mieux à les différencier et fasse moins d'erreurs lors de la reconnaissance.

D'après la collection des données, ces 36 794 données sont divisées en deux parties :

- 80% des données sont utilisées pour entraîner le modèle
- 20% des données sont utilisées pour tester le modèle

```
# Division x (données) et y (label = correction)
X = data.iloc[:, :-1] # Extraire les données de coordonnées (les 126 valeurs)
y = data.iloc[:, -1] # Extraire le label (nom du geste) -> dernière colonne
```

Figure 19

```
# Séparer l'ensemble des données augmentées en "entraînement" (80%) et "test" (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 20

Cette séparation permet de vérifier si le modèle est capable de reconnaître des gestes qu'il n'a jamais vus pendant l'apprentissage et d'éviter le surapprentissage des données.

L'accuracy correspond au pourcentage de bonnes prédictions faites par le modèle sur les données de test. Elle ne prend donc pas en compte tout le dataset mais uniquement la partie test.

La formule :

$$\bullet \text{ Accuracy} = \frac{\text{nombre de prédictions correctes}}{\text{nombre total de données de test}}$$

Comme nous avons 36 794 données au total :

- $36\,794 \times 0,2 = 7358,8$ (= nombre total de données de test)

Dans le programme, l'accuracy est calculée automatiquement par la fonction « score » par la bibliothèque « scikit-learn » :

```
# Précision (la note d'examen finale sur des données qu'il n'a pas vues)
accuracy = model.score(X_test, y_test)
print(f"Accuracy après injection de variété : {accuracy * 100:.2f}%")
```

Figure 21

Cette fonction :

1. Utilise le modèle entraîné pour prédire les gestes à partir de « X_test ».
2. Compare les prédictions avec les vraies réponses « y_test ».
3. Compte le nombre de bonnes prédictions.
4. Calcule le pourcentage de réussite.

Finalement, l'accuracy permet de mesurer la performance globale du modèle. Plus elle est proche de 100%, plus le modèle reconnaît correctement les gestes de la main.

L'accuracy obtenue dans notre projet est de 98,43% ce qui montre que le système reconnaît correctement la grande majorité des gestes. Grâce à ce résultat, nous pouvons estimer que le nombre de prédictions correctes est d'environ 7243 données d'après le calcul en appliquant la formule d'Accuracy (Nombre de prédictions correctes = $7359 \times 98,43 / 100 = 7243$).

Grâce à l'accuracy, nous pouvons évaluer la précision du modèle en observant le pourcentage obtenu lors de son entraînement.

5. Limites et perspectives d'améliorations

Il faut savoir que notre système n'est pas parfait à une précision de 100%, il peut se tromper même avec la normalisation et la data augmentation car c'est difficile d'éliminer toutes les petites variations comme le fait que deux personnes différentes vont jamais exactement avoir les mêmes coordonnées, des facteurs comme la luminosité ou l'angle de la main peuvent intervenir aussi pour créer cette différence.

Pour régler ces limites, il existe de nombreux autres algorithmes de machine learning qui pourraient être appliqués à notre projet d'après la recherche bien que l'algorithme KNN ait donné des résultats satisfaisants. Par exemple, l'algorithme SVM (Support Vector Machine) permet de trouver la meilleure séparation entre différentes classes de données ce qui est un peu similaire que KNN. Il serait intéressant de comparer ses performances avec KNN.

En plus, notre système est actuellement conçu pour reconnaître des gestes statiques. D'après la recherche, le DTW (Dynamic Time Warping) qui permet de comparer des mouvements réalisés à des vitesses différentes ou les modèles LSTM (Long Short-Term Memory) qui sont capables d'apprendre l'évolution d'un mouvement au cours du temps pourraient permettre de reconnaître des gestes dynamiques comme saluer de la main.

Conclusion

Ce premier livrable nous a permis de développer et tester un système de reconnaissance gestuelle en temps réel basé sur MediaPipe et le Machine Learning supervisé. L'application développée est capable de détecter différents gestes de la main à l'aide d'une webcam et d'afficher un émoji correspondant au geste reconnu.

Au cours de ce projet, nous avons étudié plusieurs notions telles que la détection des mains, la normalisation des données, la data augmentation et l'utilisation de l'algorithme KNN pour la classification des gestes, etc. Les différentes expérimentations réalisées ont montré des résultats satisfaisants avec une accuracy de 98,43% sur les données de test.

Le deuxième livrable portera sur l'intégration du système de reconnaissance gestuelle au robot TIAGO afin de permettre une interaction. Dans cette perspective, nous avons réalisé un premier prototype permettant de contrôler la musique par gestes (lecture, pause, musique suivante et musique précédente). Les tests réalisés sur YouTube ont donné des résultats satisfaisants et montrent que cette approche peut être développée et améliorée dans la suite du projet.